DATA 441/641 · Applied Natural Language Processing · Week 5

Lab 5: Word2Vec on Text Messages

Skip-gram, CBOW, and a t-SNE picture of SMS embeddings.

Work in a **Jupyter** notebook. Number the exercises. Do notes **5.2** through **5.5** first. You will need `pandas`, `nltk`, `gensim`, `scikit-learn`, and `matplotlib`.

Download spam.csv into the same folder as the notebook (or use a relative path). The file is the classic SMS collection: column `v1` is `ham` / `spam`, column `v2` is the message. Open it with `encoding="latin1"`.

When you are done: **File → Download as → HTML**, then upload the HTML on Canvas. Save the notebook before you export.

```
import re
import string
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from gensim.models import Word2Vec
from sklearn.manifold import TSNE

nltk.download("punkt")
nltk.download("stopwords")
```

If `word_tokenize` asks for `punkt_tab`, run `nltk.download("punkt_tab")` as well.

---

1. Load and clean (1 point)

1. Read `spam.csv` with `encoding="latin1"`. Keep `v1` and `v2`. Drop empty messages.
2. Clean each message: lowercase, strip URLs, drop punctuation and English stop words. Tokenize. Print **one** raw message next to its token list.
3. Build a list of token lists (one list per message). That is what `Word2Vec` wants.

You do not need the spam/ham label for training the embeddings. Keep it if you want to color the plot later.

---

2. Skip-gram (1 point)

Train `Word2Vec` with `sg=1` on the tokenized messages. Start with something like `vector_size=100` (or 300), `window=3`, `min_count=2`, `workers=1` so the run is repeatable.

Print the vocabulary size and the nearest neighbors of two words that actually appear (try `free`, `call`, `win`, or `ok`).

---

3. CBOW (1 point)

Train a second model on the **same** token lists with `sg=0`. Same `vector_size` and `window` as Exercise 2.

Print neighbors of the same two words. In a comment: which model's neighbors look more usable on this small SMS corpus, and why that is plausible?

---

4. t-SNE (1 point)

Take the word vectors from **one** of the models (or both, in two plots). Project them to 2-D with `TSNE(n_components=2, random_state=42)`.

If the vocabulary is large, plot a subset (for example the 200–400 most frequent words) so t-SNE finishes on a laptop. Scatter the points. Label a handful of words — and label the points that belong to **those** words, not the first few rows of the matrix.

```
vectors = model.wv[words]
coords = TSNE(n_components=2, random_state=42, init="pca", perplexity=30).fit_transform(vectors)
```

---

5. Read the picture (1 point)

Write a short paragraph (a few sentences is enough):

- Do you see any clusters (greeting words, prize/spam language, numbers)?
- Name two words that landed near each other and whether that pairing makes sense.
- One limitation of training Word2Vec on a few thousand short SMS texts.